FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS
ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS

Auditoría de Tecnologías de la Información

AGENTE DE CONTROLES Y SOA PARA PIMS ISO/IEC 27701:2025

TIF – Primera Parte: De Prompt a Skill en Sistemas Multi-Agente

AUTORES:
Ajra Huasco Jeans Anthony
Gutierrez Ccam Juan Diego
Suclle Suca Michael Benjamin
Garambel Marin Fernando Miguel
Hanco Mullisaca Sergio Danilo

DOCENTE:
Docente de la Asignatura

CURSO:
Auditoría de Tecnologías de la Información

LUGAR:
Arequipa, Perú

FECHA:
01 de Octubre de 2026

Índice

1 Introducción y Marco General . . . 1
1.1 Introducción . . . 1
1.2 Objetivos del Proyecto . . . 1
1.2.1 Objetivo General . . . 1
1.2.2 Objetivos Específicos . . . 1
1.3 Contexto Normativo: Estructura Armonizada e ISO/IEC 27701:2025 . . . 2
1.3.1 Estructura de Alto Nivel (Cláusulas 1 a 10) . . . 2
1.3.2 Roles Organizacionales y Estructura del Anexo A . . . 3
1.3.3 El Rol del DPO y la Gobernanza . . . 3
1.4 El Ecosistema Multi-Agente: Arquitectura en Dos Pisos . . . 3
2 Desarrollo Metodológico e Implementación . . . 4
2.1 De Prompt a Skill: Transformación y Metodología . . . 4
2.1.1 Transformación del Caso 1: Elaboración de la SoA . . . 4
2.1.2 Transformación del Caso 2: Auditoría de un Control Específico . . . 4
2.2 Especificación Formal de los Skills . . . 4
2.2.1 Ficha Técnica del Skill `generar-soa` . . . 4
2.2.2 Ficha Técnica del Skill `verificar-controles` . . . 5
2.3 Definición y Modelado del Asistente . . . 5
2.4 Arquitectura del Sistema y Contratos de Integración . . . 6
2.4.1 Contrato Estándar de Comunicación Inter-Agentes . . . 6
2.4.2 Flujo de Ejecución e Interacción Temporal . . . 7
2.5 Tecnologías, Motor Determinista e Infraestructura . . . 7
2.6 Demostración y Plan de Pruebas . . . 8
3 Conclusiones y Trabajo Futuro . . . 8
3.1 Conclusiones . . . 8
3.2 Limitaciones . . . 9
3.3 Hoja de Ruta y Recomendaciones para la Entrega 2 . . . 9
Referencias Bibliográficas . . . 10
4 Anexos . . . 10
4.1 Anexo A: Catálogo de Controles de Privacidad (`catalogo_controles.json`) . . . 10
4.2 Anexo B: Especificación de Archivos `SKILL.md` . . . 11
4.2.1 Skill `generar-soa` (`skills/generar-soa/SKILL.md`) . . . 11
4.2.2 Skill `verificar-controles` (`skills/verificar-controles/SKILL.md`) . . . 12
4.3 Anexo C: Código Fuente Base . . . 13
4.3.1 Módulo RAG Local (`rag.py`) . . . 13
4.3.2 Agente de Controles (`agents/controles.py`) . . . 14

1 Introducción y Marco General

1.1 Introducción

El aseguramiento de la privacidad y la protección de los Datos de Identificación Personal (PII, *Personally Identifiable Information*) se han consolidado como pilares regulatorios, operacionales y éticos en la era digital. En este contexto, la norma internacional **ISO/IEC 27701:2025** [1] establece los requisitos y directrices para implementar, mantener y auditar un Sistema de Gestión de Información de Privacidad (PIMS, *Privacy Information Management System*), actuando como una extensión armónica de las normas ISO/IEC 27001:2022 [2] e ISO/IEC 27002, alineándose además con marcos normativos de alcance global como el Reglamento General de Protección de Datos (GDPR) [3] y los estándares emergentes de gobernanza de inteligencia artificial como ISO/IEC 42001:2023 [4].

Dentro del ciclo de vida de un PIMS, la **Declaración de Aplicabilidad (SoA, *Statement of Applicability*)** representa el instrumento técnico y legal más crítico. En ella se documenta formalmente qué controles normativos aplican a la organización, cuáles se excluyen, la justificación preceptiva de cada decisión y el estado real y verificable de su implementación. No obstante, la gestión tradicional de la SoA suele sufrir de dispersión probatoria, inconsistencias humanas y una alta carga burocrática.

Para superar estas limitaciones, este proyecto aborda la transición paradigmática **De Prompt a Skill** en el marco de una arquitectura de **Sistemas Multi-Agente (MAS, *Multi-Agent Systems*)**. Mientras que un prompt convencional es una instrucción no estructurada, reactiva y propensa a alucinaciones, un **Skill** es una unidad de procedimiento determinista, modular, reusable y gobernada por contratos de datos tipados. En este informe se detalla el diseño y construcción del **Agente de Controles**, el componente articulador que sintetiza la información de todo el sistema para emitir la SoA y auditar el cumplimiento normativo.

1.2 Objetivos del Proyecto

1.2.1 Objetivo General

Diseñar, implementar y validar experimentalmente el **Agente de Controles** para un PIMS bajo la norma ISO/IEC 27701:2025, transformando tareas de auditoría en **Skills especializados deterministas** e interoperables mediante contratos JSON estándar en una arquitectura multi-agente.

1.2.2 Objetivos Específicos

- **Formalizar la metodología «De Prompt a Skill»:** Transformar consultas abiertas y ad-hoc en procedimientos reproducibles y tipados (`generar-soa` y `verificar-controles`).
- **Garantizar la auditabilidad y mitigación de alucinaciones:** Implementar un motor de reglas determinista que impida declarar controles implementados sin evidencias objetivas y verificables.

- **Integrar conocimiento semántico local (RAG):** Implementar un índice vectorial con embeddings multilingües y FAISS para vincular políticas internas y cláusulas de la norma sin dependencia de servicios externos.
- **Establecer el protocolo de interoperabilidad inter-agentes:** Diseñar el contrato estándar JSON para la comunicación horizontal con los otros 7 agentes del PIMS y la integración vertical con el orquestador.
- **Establecer las bases para la Entrega 2:** Diseñar una arquitectura modular capaz de evolucionar hacia la orquestación distribuida, contenedores Docker y validación con datos sintéticos y entornos reales.

1.3 Contexto Normativo: Estructura Armonizada e ISO/IEC 27701:2025

1.3.1 Estructura de Alto Nivel (Cláusulas 1 a 10)

Al igual que los estándares modernos de gestión (como ISO/IEC 27001 e ISO/IEC 42001), la norma ISO/IEC 27701:2025 adopta la Estructura Armonizada (*Harmonized Structure* / HLS), compuesta por 10 cláusulas fundamentales:

CláusulaDenominación y AlcanceRelevancia en el PIMS / Agente

1. Alcance

Delimitación del PIMS y tipos de organizaciones cubiertasDefine si la entidad es responsable o encargada

2. Normas de referencia

Vínculo mandatorio con ISO/IEC 27001 e ISO/IEC 27002Marco de seguridad base

3. Términos y definiciones

Conceptos clave: PII, titular de datos, DPO, brechaVocabulario formal del sistema

4. Contexto

Comprensión de la organización, partes interesadas y PIIEntrada del Agente de Inventario

5. Liderazgo

Compromiso de la alta dirección y política de privacidadMarco de gobernanza institucional

6. Planificación

6.1.2 Evaluación de riesgos; 6.1.3 Tratamiento de riesgos / SoA**Núcleo del Agente de Controles y Riesgos**

7. Soporte

Recursos, competencia (7.2), concienciación (7.3) e info. doc. (7.5)Agente de Formación y Repositorio

8. Operación

Planificación y control operacional; evaluaciones DPIAAgente de Riesgos y Terceros

9. Evaluación de desempeño

9.1 Seguimiento y medición; 9.2 Auditoría interna; 9.3 Revisión**Base probatoria del Agente de Evidencia**

10. Mejora

No conformidades, acciones correctivas y mejora continuaAgente de Incidentes y Planes de acción

1.3.2 Roles Organizacionales y Estructura del Anexo A

La norma ISO/IEC 27701:2025 exige diferenciar explícitamente el rol que asume la organización:

1. **Responsable del Tratamiento (*PII Controller*):** Determina las finalidades y medios del tratamiento de datos personales. Está sujeto a los **Controles A.1** (gobierno, finalidades, derechos de titulares y DPIA).
2. **Encargado del Tratamiento (*PII Processor*):** Trata datos exclusivamente bajo instrucciones del responsable. Está sujeto a los **Controles A.2** (gestión de subencargados, asistencia técnica y transferencias).
3. **Controles Comunes (A.3):** Controles transversales de seguridad operacional, respuesta a incidentes y concienciación, obligatorios para **ambos roles**.

1.3.3 El Rol del DPO y la Gobernanza

El estándar contempla la figura del **Oficial de Protección de Datos (DPO, *Data Protection Officer*)** como garante del cumplimiento normativo. En nuestra arquitectura, el sistema automatizado actúa como un asistente analítico avanzado; por tanto, la SoA emitida ostenta carácter de **propuesta técnica auditada**, requiriendo preceptivamente la bandera de supervisión y aprobación humana (`requiere_revision_humana = true`).

1.4 El Ecosistema Multi-Agente: Arquitectura en Dos Pisos

Para abordar la complejidad de un PIMS corporativo, la solución se concibe en dos niveles arquitectónicos:

- **Primer Piso (Capa Tradicional / Transaccional):** Modelado de datos, tablas maestras de sistemas, repositorios de políticas, registros de actividades de tratamiento (RAT) y almacenamiento de evidencias (similar a las plataformas tradicionales de cumplimiento y GDPR).
- **Segundo Piso (Capa Agéntica Multi-Agente):** Capa de razonamiento compuesta por 8 agentes especializados que se comunican mediante protocolos estándar (JSON / MCP - *Model Context Protocol*), automatizando el cruce de requisitos, riesgos y evidencias.

AgenteRol y ResponsabilidadCláusula ISOAporte e Interacción InventarioDescubre sistemas, cataloga PII y propósitos4.1, 4.2, A.1.2.2Provee el objeto `perfil` y contextoRiesgosEjecuta DPIAs y análisis de riesgos6.1.2, 8.2, A.1.2.6Provee el arreglo `riesgos`**Controles (Nuestro)Evalúa aplicabilidad, cruza evidencias y genera SoA6.1.3, Anexo ASíntesis, SoA y Matriz de Brechas**EvidenciaRecopila artefactos probatorios y logs9.1, 9.2, 7.5Provee el arreglo `evidencias`Derechos (DSAR)Gestiona solicitudes de acceso y rectificaciónA.1.3.7, A.1.3.9Evidencias de ejercicio de derechosIncidentesClasifica, notifica y gestiona brechas (Ishikawa)A.3.11, A.3.12, 10.1Evidencias de gestión de incidentesTercerosAudita contratos DPA y subencargadosA.1.2.7, A.2.5.7Evidencias de proveedores y DPAsFormaciónMonitorea la capacitación y cultura de privacidad7.2, 7.3, A.3.17Evidencias de sensibilización anual

El **Agente de Controles «cierra el círculo del PIMS»:** toma los riesgos identificados, las actividades inventariadas y las pruebas recopiladas por los demás agentes para emitir la SoA definitiva y señalar las brechas de cumplimiento con sus respectivos responsables.

2 Desarrollo Metodológico e Implementación

2.1 De Prompt a Skill: Transformación y Metodología

En las etapas iniciales de la adopción de inteligencia artificial en auditoría, las tareas de verificación suelen abordarse mediante **prompts** empíricos enviados directamente a un modelo de lenguaje. Sin embargo, en un entorno de cumplimiento regulatorio e inspección de seguridad, los prompts presentan debilidades estructurales críticas:

DimensiónEnfoque con Prompt TradicionalEnfoque con Skill Especializado DeterminismoComportamiento estocástico; respuestas variables ante una misma entradaEjecución algorítmica con reglas de negocio fijas e invariantesEstructura de DatosTexto libre no tipado, dependiente de la formulación del promptContrato JSON estricto con validación de esquemas y tiposRigor ProbatorioPropenso a alucinar «implementado» por cortesía conversacionalImposibilidad de asignar «implementado» sin evidencias verificablesTrazabilidadOmite justificaciones o inventa motivos no normativosJustificación formal obligatoria vinculada al rol y al catálogoReusabilidadRequiere reescribir y refinar la instrucción en cada consultaInvocación modular estandarizada por humanos o por el orquestador

2.1.1 Transformación del Caso 1: Elaboración de la SoA

- **Prompt Original:** *«Con esta información de mi organización, dime qué controles de ISO 27701 aplican, por qué, y en qué estado de implementación están.»*
- **Skill Resultante (`generar-soa`):** Procedimiento formal que valida el rol (`responsable`, `encargado`, `ambos`), analiza los riesgos vinculados, computa el estado de implementación a partir de la matriz de evidencias, consulta políticas en FAISS y emite la SoA completa junto al resumen de brechas y la bandera obligatoria de supervisión humana (`requiere_revision_humana = true`).

2.1.2 Transformación del Caso 2: Auditoría de un Control Específico

- **Prompt Original:** *«Revisa si el control de DPIA está realmente implementado con estas evidencias y dime qué falta.»*
- **Skill Resultante (`verificar-controles`):** Procedimiento especializado que valida la existencia del control en el catálogo normativo, computa la brecha exacta y formula una recomendación prescriptiva indicando qué agente específico debe subsanar la deficiencia.

2.2 Especificación Formal de los Skills

Los Skills se definen siguiendo el estándar de especificación de agentes, almacenados en archivos `SKILL.md` dentro del directorio `skills/`.

2.2.1 Ficha Técnica del Skill `generar-soa`

- **Nombre:** `generar-soa`
- **Descripción:** Elabora la Declaración de Aplicabilidad (SoA) integral conforme a ISO/IEC 27701:2025.
- **Condición de Activación:** Peticiones de elaboración, actualización o auditoría completa de la SoA organizacional.
- **Entradas:**

- `perfil`: `{ organizacion: string, rol: "responsable" | "encargado" | "ambos" }`
- `riesgos`: Lista de `{ id: string, descripcion: string, controles: string[] }`
- `evidencias`: Lista de `{ control: string, descripcion: string, estado: "implementado" | "parcial" }`

- **Algoritmo de Ejecución:**
  1. Validar la obligatoriedad y formato del rol. Si es inválido, abortar con estado `"rechazada"`.
  2. Cargar el catálogo normativo oficial (`catalogo_controles.json`).
  3. Evaluar aplicabilidad: controles `A.1` (requieren rol responsable o ambos), `A.2` (encargado o ambos), `A.3` (universal).
  4. Redactar justificación técnica; asociar IDs de riesgos del Agente de Riesgos si aplica.
  5. Computar el estado estrictamente en base a evidencias recibidas: `implementado`, `parcial`, `no implementado` o `no aplica`.
  6. Consultar mediante embeddings y FAISS la política organizacional vinculada.
  7. Identificar brechas y asignar el agente responsable de su subsanación.
  8. Consolidar totales estadísticos y marcar obligatoriamente `requiere_revision_humana = true`.
- **Restricciones Clave:** Prohibición absoluta de declarar controles implementados sin evidencias documentadas; justificación obligatoria para toda exclusión; emisión en calidad de propuesta técnica sin auto-aprobación.

2.2.2 Ficha Técnica del Skill `verificar-controles`

- **Nombre:** `verificar-controles`
- **Descripción:** Evalúa el estado de cumplimiento de un control normativo puntual e identifica acciones correctivas.
- **Entradas:** `control` (ID normativo, ej. `"A.1.2.6"`), `perfil` (`rol`), `riesgos` y `evidencias`.
- **Salida:** Fila estructurada de control con diagnóstico detallado y recomendación prescriptiva dirigida al agente correspondiente.

2.3 Definición y Modelado del Asistente

El Asistente de Controles se modela bajo un principio de especialización estricta:

- **Identidad Sistémica:** *«Soy el Agente de Controles del sistema de gestión de privacidad (PIMS) basado en ISO/IEC 27701:2025. Mi función es determinar qué controles del Anexo A aplican a la organización, generar la Declaración de Aplicabilidad (SoA) y verificar el estado de implementación de cada control.»*
- **Directrices Operativas:** Priorizar la ejecución de los Skills estructurados; no realizar inferencias especulativas sobre la existencia de documentos no provistos; rechazar entradas con datos incompletos solicitando la aclaración de perfil; mantener estricta neutralidad y trazabilidad auditable.
- **Base de Conocimiento Local:**

- `knowledge/catalogo_controles.json`: Catálogo de controles con metadatos y agente responsable.
- `knowledge/normativa/resumen_27701.md`: Síntesis autorizada de cláusulas de ISO/IEC 27701:2025.
- `knowledge/politicas/politica_privacidad_ejemplo.md`: Repositorio corporativo de políticas de privacidad.

2.4 Arquitectura del Sistema y Contratos de Integración

La solución se estructura mediante una arquitectura desacoplada basada en microservicios y protocolos JSON, preparada para su despliegue individual o coordinado mediante un bus de integración.

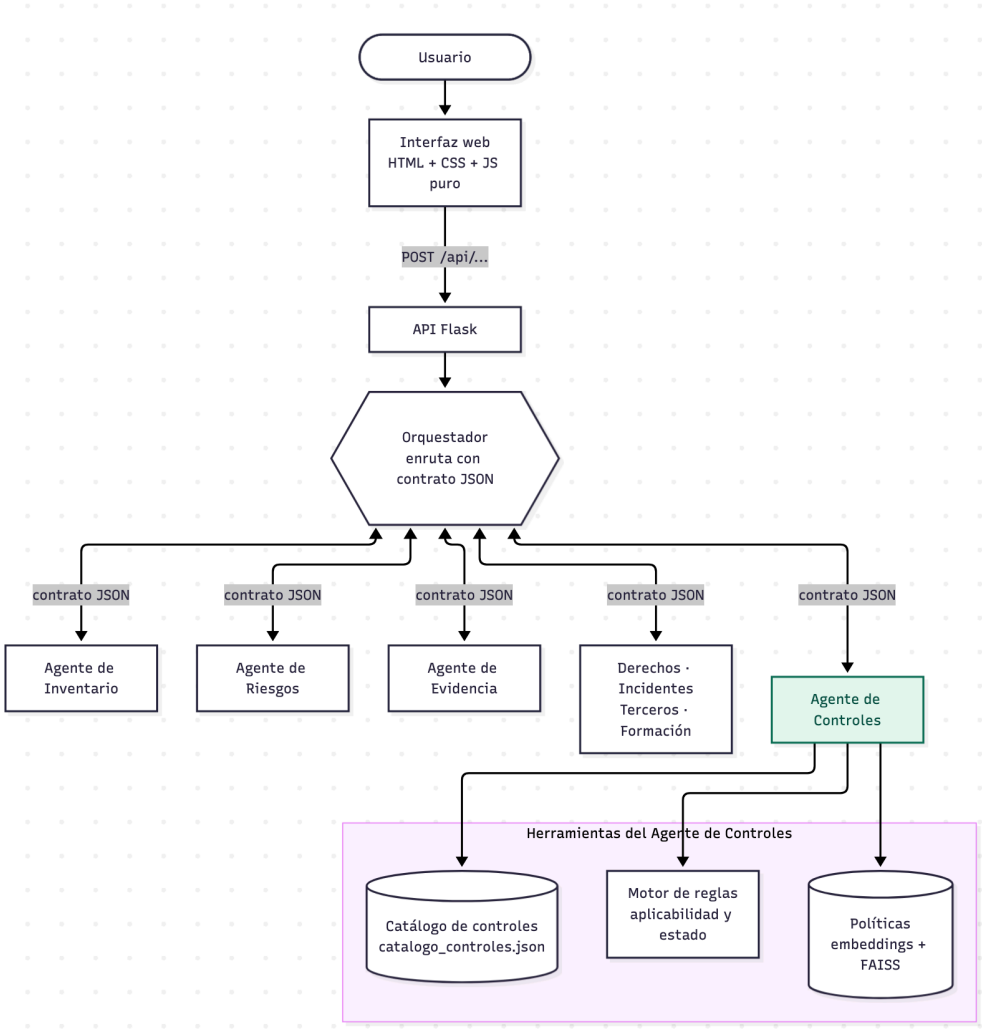


Figura 1: Arquitectura de Componentes del Ecosistema PIMS y el Agente de Controles.

2.4.1 Contrato Estándar de Comunicación Inter-Agentes

Para garantizar la interoperabilidad entre los 8 agentes del PIMS, se estableció una estructura JSON uniforme:

```
{
  "agente": "controles",
```

```
  "operacion": "generar_soa | verificar_control | consultar",
  "estado": "ok | rechazada | sin_respuesta",
  "respuesta": "Resumen ejecutivo en lenguaje natural",
  "datos": {
    "soa": [ /* Matriz de controles evaluados */ ],
    "brechas": [ /* Controles aplicables no implementados con
responsable */ ],
    "resumen": { /* Métricas agregadas */ }
  },
  "fuentes": ["catalogo_controles.json",
"politica_privacidad_ejemplo.md"],
  "confianza": 1.0,
  "requiere_revision_humana": true
}
```

2.4.2 Flujo de Ejecución e Interacción Temporal

El procesamiento de la solicitud de generación de SoA sigue una secuencia coordinada entre el usuario, el servidor web, el orquestador y los agentes:

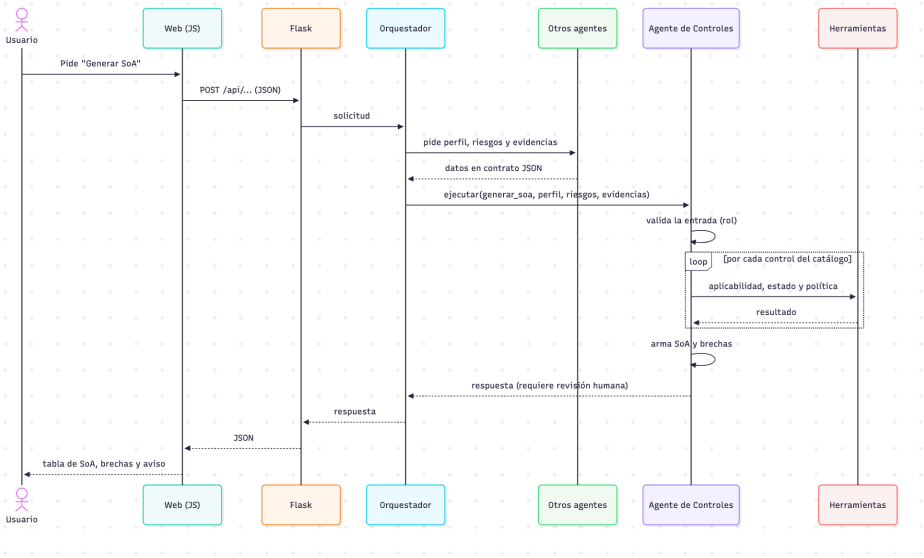


Figura 2: Diagrama de Secuencia del flujo de generación y verificación de la SoA.

2.5 Tecnologías, Motor Determinista e Infraestructura

La implementación técnica adopta un enfoque híbrido que balancea determinismo, privacidad de datos y eficiencia computacional:

1. **Lenguaje Backend y Servidor Web:** Desarrollado en **Python 3** utilizando el microframework **Flask** para exponer endpoints REST (`/api/controles/ejecutar` y `/api/controles/catalogo`).
2. **Motor de Reglas Determinista:** Métodos puros `_aplica()` y `_estado()` en `AgenteControles`. A diferencia de los modelos generativos estocásticos, las reglas lógicas garantizan que la evaluación del Anexo A sea 100% reproducible y auditable, eliminando el riesgo de alucinaciones en decisiones regulatorias.

3. **Motor de Embeddings Local:** Empleo de `SentenceTransformer` con el modelo multilingüe `paraphrase-multilingual-MiniLM-L12-v2` [5], que procesa semántica en español de forma 100% offline en CPU local sin latencia de red ni costos de API externa.
4. **Indexación Vectorial con FAISS:** Utilización de `faiss-cpu` [6] con métrica de producto interno (`IndexFlatIP`) sobre vectores normalizados para búsqueda de similitud coseno ultrarrápida en memoria.
5. **Integración Opcional de SLMs/LLMs:** En consultas conceptuales (`_consultar`), el sistema permite conectar un modelo de lenguaje local (e.g. Ollama con Llama 3 o Mistral) para enriquecer la redacción, restringiendo su contexto a los fragmentos recuperados por FAISS, manteniendo las reglas de la SoA estrictamente en código.
6. **Estrategia de Despliegue e Infraestructura:** Diseñado para ejecutarse en computadoras de laboratorio o desplegarse mediante contenedores **Docker**, permitiendo que las instancias de los diferentes grupos interactúen de forma homogénea en una red local o en servidores virtuales (VPS) asegurando soberanía de datos.

2.6 Demostración y Plan de Pruebas

Se diseñó una batería de 6 casos de prueba exhaustivos para validar la integridad del agente:

N°Caso de PruebaDatos de EntradaResultado Esperado y Verificado1Generación SoA (Responsable)Rol: `responsable`, Riesgo: `R1`, Evidencias: `A.1.2.2` (imp.), `A.3.12` (parcial)10 evaluados, 9 aplican, 1 excluido (`A.2.5.7`), 1 implementado, 1 parcial, 7 no implementados. 8 brechas detectadas. Flag `requiere_revision_humana: true`.2Cambio de Rol a EncargadoRol: `encargado`, mismas evidenciasControles `A.1` quedan excluidos con justificación formal; aplican `A.2.5.7` y controles comunes `A.3`.3Cambio de Rol a AmbosRol: `ambos`Los 10 controles del catálogo aplican en su totalidad con sus respectivas justificaciones.4Verificación Puntual (A.1.2.6)Control: `A.1.2.6`, Rol: `responsable`, sin evidenciasEstado: `no implementado`. Recomendación prescriptiva: «Solicitar evidencia o implementación al Agente de Riesgos».5Consulta Normativa RAGPregunta: «¿Qué debe incluir la Declaración de Aplicabilidad?»Recuperación semántica de fragmentos de `resumen_27701.md`, fuentes citadas y nivel de confianza calculado.6Validación de Entrada ErróneaEntrada sin campo `perfil.rol` o rol inválidoRechazo inmediato con estado `"rechazada"` y mensaje descriptivo de error, sin fallos de ejecución.

3 Conclusiones y Trabajo Futuro

3.1 Conclusiones

A partir del diseño, implementación y validación experimental del **Agente de Controles** para el Sistema de Gestión de Información de Privacidad (PIMS) bajo ISO/IEC 27701:2025, se concluye lo siguiente:

1. **Superación del paradigma de prompts no estructurados:** La transición **«De Prompt a Skill»** resulta imperativa en la ingeniería de software aplicada a la auditoría de sistemas. Al encapsular la lógica en procedimientos formales (`generar-soa` y `verificar-controles`), se transforma una interacción estocástica e informal en un proceso algorítmico, determinista, tipado y 100% reproducible.

2. **Garantía de auditabilidad e inmunidad a alucinaciones:** La adopción de una arquitectura híbrida —que confía las decisiones de aplicabilidad y estado a un motor de reglas determinista y la vinculación contextual a un índice vectorial local (FAISS)— asegura que ningún control sea declarado falsamente como «implementado» sin evidencias probatorias objetivas, satisfaciendo el estándar de las cláusulas 6.1.3 y 9.2 de la norma.
3. **Articulación del ecosistema multi-agente:** El Agente de Controles cumple con éxito su función como elemento de cierre del ciclo del PIMS. Mediante el contrato estándar JSON, recibe los insumos de Inventario, Riesgos y Evidencias, generando de manera unificada la SoA corporativa y la matriz prescriptiva de brechas.
4. **Preservación del principio de supervisión humana y gobernanza:** La inclusión del atributo mandatorio `requiere_revision_humana = true` garantiza el cumplimiento ético y regulatorio del sistema. La IA actúa como un instrumento de diagnóstico y auditoría asistida, reservando la aprobación final de la SoA a la alta dirección y al Oficial de Protección de Datos (DPO).

3.2 Limitaciones

- **Catálogo inicial acotado:** La presente entrega incorpora 10 controles representativos del Anexo A para validar la arquitectura funcional; el estándar completo en un entorno corporativo de producción abarca más de 30 controles y subcontroles.
- **Calibración de umbrales vectoriales:** Los coeficientes de similitud coseno (0.45 para políticas y 0.30 para consultas normativas) requieren ajuste continuo conforme crezca el volumen y variedad de la base documental.

3.3 Hoja de Ruta y Recomendaciones para la Entrega 2

Con miras a la segunda entrega del proyecto integral del curso, se plantean las siguientes líneas de desarrollo:

- **Ampliación Integral del Catálogo Normativo:** Incorporar la totalidad de controles del Anexo A de ISO/IEC 27701:2025, cubriendo la totalidad de directrices para responsables y encargados.
- **Orquestación y Despliegue en Red (Docker):** Desplegar los 8 agentes en contenedores Docker independientes intercomunicados a través de un bus de mensajería (o protocolo MCP), permitiendo la interoperabilidad unificada entre los grupos de trabajo en servidores locales o instancias en la nube.
- **Integración con Datos Sintéticos y Bases de Datos:** Conectar la capa agéntica («segundo piso») con la base de datos relacional transaccional («primer piso»), poblando el sistema con datos sintéticos representativos de organizaciones de sectores de alta criticidad (empresas de energía/utilities, universidades y entidades financieras).
- **Capa de Generación Asistida con SLMs Locales:** Integrar un modelo de lenguaje local ligero (e.g. Ollama con modelos cuantizados) exclusivamente para la redacción de informes ejecutivos y explicaciones pedagógicas guiadas por RAG, blindando las reglas de la SoA en código determinista.

Referencias Bibliográficas

[1] «ISO/IEC 27701:2025 Security techniques – Extension to ISO/IEC 27001 and ISO/IEC 27002 for privacy information management – Requirements and guidelines». Geneva, Switzerland, 2025.

[2] «ISO/IEC 27001:2022 Information security, cybersecurity and privacy protection – Information security management systems – Requirements». Geneva, Switzerland, 2022.

[3] European Parliament and Council of the European Union, «Regulation (EU) 2016/679 of the European Parliament and of the Council on the protection of natural persons with regard to the processing of personal data (GDPR)», *Official Journal of the European Union*, vol. L119, pp. 1-88, 2016.

[4] «ISO/IEC 42001:2023 Information technology – Artificial intelligence – Management system». Geneva, Switzerland, 2023.

[5] N. Reimers y I. Gurevych, «Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks», *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 3982-3992, 2019.

[6] J. Johnson, M. Douze, y H. Jégou, «Billion-scale similarity search with GPUs», *IEEE Transactions on Big Data*, vol. 7, n.º 3, pp. 535-547, 2019.

[7] I. Sommerville, *Software Engineering*, 9.ª ed. Addison-Wesley, 2011.

[8] R. S. Pressman y B. R. Maxim, *Software Engineering: A Practitioner's Approach*, 8.ª ed. McGraw-Hill Education, 2014.

[9] P. Lewis *et al.*, «Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks», *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, pp. 9459-9474, 2020.

4 Anexos

4.1 Anexo A: Catálogo de Controles de Privacidad (`catalogo_controles.json`)

El catálogo normativo base mapea cada control del Anexo A con el agente especializado encargado de su gestión:


```
[
  {"id": "A.1.2.2", "nombre": "Identificar propósito", "agente_responsable":
"Agente de Inventario"},
  {"id": "A.1.2.9", "nombre": "Registros de tratamiento",
"agente_responsable": "Agente de Inventario"},
  {"id": "A.1.2.6", "nombre": "Evaluación de impacto de privacidad (DPIA)",
"agente_responsable": "Agente de Riesgos"},
  {"id": "A.1.2.7", "nombre": "Contratos con encargados",
"agente_responsable": "Agente de Terceros"},
  {"id": "A.1.3.7", "nombre": "Acceso, corrección o supresión",
"agente_responsable": "Agente de Derechos"},
  {"id": "A.1.3.9", "nombre": "Copia de PII", "agente_responsable": "Agente de
Derechos"},
  {"id": "A.2.5.7", "nombre": "Divulgación de subcontratistas",
```

```
"agente_responsable": "Agente de Terceros"},
  {"id": "A.3.11", "nombre": "Planificación de incidentes",
"agente_responsable": "Agente de Incidentes"},
  {"id": "A.3.12", "nombre": "Respuesta a incidentes", "agente_responsable":
"Agente de Incidentes"},
  {"id": "A.3.17", "nombre": "Concienciación y formación",
"agente_responsable": "Agente de Formación"}
]
```

4.2 Anexo B: Especificación de Archivos `SKILL.md`

4.2.1 Skill `generar-soa` (`skills/generar-soa/SKILL.md`)

```
---
name: generar-soa
description: Genera la Declaración de Aplicabilidad (SoA) del PIMS
según ISO/IEC
  27701:2025. Para cada control indica si aplica, la justificación, el
estado de
  implementación y las brechas. Usar cuando se pida elaborar,
actualizar o revisar
  la SoA.
---

# Generar la Declaración de Aplicabilidad (SoA)

## Activación
Se ejecuta cuando:
- Se solicita elaborar o actualizar la SoA de la organización.
- Otro agente (Riesgos, Evidencia) entrega información nueva que puede
cambiar
  la aplicabilidad o el estado de los controles.

No se ejecuta para: consultas sobre un solo control (usar `verificar-
controles`).

## Entrada
- perfil: { organizacion, rol } con rol = responsable | encargado |
ambos
- riesgos: lista de { id, descripcion, controles[] }        (de Agente
de Riesgos)
- evidencias: lista de { control, descripcion, estado }      (de Agente
de Evidencia)
  con estado = implementado | parcial

## Pasos
1. Validar la entrada. Si falta perfil.rol o no es válido, rechazar con
estado "rechazada".
2. Cargar el catálogo de controles (control, nombre, agente
responsable).
3. Determinar la aplicabilidad de cada control según el rol:
   - A.1 (responsable): aplica si rol = responsable o ambos.
```

```
   - A.2 (encargado): aplica si rol = encargado o ambos.
   - A.3 (común): aplica siempre.
4. Redactar la justificación:
   - Si aplica: motivo + riesgos vinculados (si los hay).
   - Si no aplica: motivo de la exclusión (obligatorio).
5. Determinar el estado de implementación SOLO con evidencias:
   - implementado: existe al menos una evidencia "implementado".
   - parcial: solo hay evidencias "parcial".
   - no implementado: no hay evidencias.
   - no aplica: control excluido.
6. Buscar en el repositorio de políticas (búsqueda semántica) una
política relacionada.
7. Identificar brechas: controles aplicables no implementados,
indicando a qué agente pedir evidencia.
8. Armar la SoA y el resumen con totales estadísticos.
9. Marcar requiere_revision_humana = true.

## Salida
{
  "agente": "controles",
  "operacion": "generar_soa",
  "estado": "ok | rechazada",
  "respuesta": "resumen en texto",
  "datos": { "soa": [ ... ], "brechas": [ ... ], "resumen": { ... } },
  "fuentes": ["catalogo_controles.json"],
  "confianza": 1.0,
  "requiere_revision_humana": true
}
```

4.2.2 Skill `verificar-controles` (`skills/verificar-controles/SKILL.md`)

```
---
name: verificar-controles
description: Verifica el estado de implementación de un control
concreto del PIMS
  (ISO/IEC 27701:2025), identifica la brecha y recomienda la siguiente
acción.
---

# Verificar un control

## Activación
Se ejecuta cuando se pide revisar un control específico o saber qué
falta para cumplirlo.

## Entrada
- control: identificador (por ejemplo "A.1.2.6")
- perfil: { rol }
- riesgos y evidencias (mismo formato que en generar-soa)

## Pasos
1. Validar existencia del control y rol válido.
2. Determinar si el control aplica al rol.
```

```
3. Reunir las evidencias del control.
4. Buscar una política relacionada en el repositorio FAISS.
5. Determinar estado (implementado / parcial / no implementado / no
aplica).
6. Recomendar la siguiente acción indicando el agente responsable.
7. Devolver el resultado en el contrato estándar.
```

4.3 Anexo C: Código Fuente Base

4.3.1 Módulo RAG Local (rag.py)

```python
import glob
import os
from functools import lru_cache
import faiss
import numpy as np
from sentence_transformers import SentenceTransformer

@lru_cache(maxsize=1)
def get_model():
    return SentenceTransformer("paraphrase-multilingual-MiniLM-L12-v2")

class Indice:
    def __init__(self, textos, metas):
        self.textos, self.metas = textos, metas
        self.index = None
        if not textos:
            return
        vecs = get_model().encode(textos, normalize_embeddings=True)
        self.index = faiss.IndexFlatIP(vecs.shape[1])
        self.index.add(np.asarray(vecs, dtype="float32"))

    def buscar(self, consulta, k=3):
        if self.index is None:
            return []
        q = np.asarray(get_model().encode([consulta],
normalize_embeddings=True), dtype="float32")
        scores, ids = self.index.search(q, min(k, len(self.textos)))
        return [
            {"texto": self.textos[i], "meta": self.metas[i], "score":
float(s)}
            for s, i in zip(scores[0], ids[0]) if i != -1
        ]

def desde_carpeta(carpeta):
    textos, metas = [], []
    for ruta in sorted(glob.glob(os.path.join(carpeta, "*.md"))):
        with open(ruta, encoding="utf-8") as f:
            for parte in f.read().split("\n\n"):
                if parte.strip():
                    textos.append(parte.strip())
```

```
                    metas.append(os.path.basename(ruta))
    return Indice(textos, metas)
```

4.3.2 Agente de Controles (`agents/controles.py`)

```
import json
import os
from rag import desde_carpeta

ROLES = ("responsable", "encargado", "ambos")
UMBRAL_POLITICA = 0.45
UMBRAL_CONSULTA = 0.30

class AgenteControles:
    nombre = "controles"
    descripcion = "Genera la SoA y verifica la implementación de
controles de ISO/IEC 27701:2025."

    def __init__(self, carpeta="knowledge"):
        with open(os.path.join(carpeta, "catalogo_controles.json"),
encoding="utf-8") as f:
            self.catalogo = json.load(f)
        self.normativa = desde_carpeta(os.path.join(carpeta,
"normativa"))
        self.politicas = desde_carpeta(os.path.join(carpeta,
"politicas"))

    def ejecutar(self, entrada):
        operacion = entrada.get("operacion", "consultar")
        try:
            if operacion == "generar_soa":
                return self._generar_soa(entrada)
            if operacion == "verificar_control":
                return self._verificar_control(entrada)
            return self._consultar(entrada.get("solicitud", ""))
        except ValueError as e:
            return self._resp(operacion, "rechazada", f"Entrada
inválida: {e}")

    @staticmethod
    def _aplica(control, rol):
        ambito = control["id"][:3]
        if ambito == "A.3":
            return True, "Control común de seguridad: aplica a
cualquier rol."
        if ambito == "A.1":
            if rol in ("responsable", "ambos"):
                return True, "La organización actúa como responsable
del tratamiento."
            return False, "Excluido: la organización no actúa como
responsable (controles A.1)."
        if ambito == "A.2":
            if rol in ("encargado", "ambos"):
```

```
                return True, "La organización actúa como encargada del
tratamiento."
            return False, "Excluido: la organización no actúa como
encargada (controles A.2)."
        raise ValueError(f"ámbito desconocido en el control
{control['id']}")

    @staticmethod
    def _estado(control_id, evidencias):
        propias = [e for e in evidencias if e.get("control") ==
control_id]
        if any(e.get("estado") == "implementado" for e in propias):
            return "implementado", propias
        if any(e.get("estado") == "parcial" for e in propias):
            return "parcial", propias
        return "no implementado", propias

    def _fila(self, control, rol, riesgos, evidencias):
        aplica, justificacion = self._aplica(control, rol)
        vinculados = [r["id"] for r in riesgos if control["id"] in
r.get("controles", [])]
        if aplica and vinculados:
            justificacion += f" Vinculado a los riesgos: {',
'.join(vinculados)}."

        if aplica:
            estado, propias = self._estado(control["id"], evidencias)
        else:
            estado, propias = "no aplica", []

        hits = self.politicas.buscar(control["nombre"], k=1)
        politica = hits[0]["meta"] if hits and hits[0]["score"] >=
UMBRAL_POLITICA else None

        return {
            "control": control["id"],
            "nombre": control["nombre"],
            "aplica": aplica,
            "justificacion": justificacion,
            "estado": estado,
            "evidencias": [e.get("descripcion", "") for e in propias],
            "politica_relacionada": politica,
            "responsable": control["agente_responsable"],
        }

    def _generar_soa(self, entrada):
        perfil = entrada.get("perfil") or {}
        rol = perfil.get("rol")
        if rol not in ROLES:
            raise ValueError("perfil.rol debe ser 'responsable',
'encargado' o 'ambos'")
        riesgos = entrada.get("riesgos", [])
```

```
        evidencias = entrada.get("evidencias", [])

        soa = [self._fila(c, rol, riesgos, evidencias) for c in
self.catalogo]
        aplicables = [f for f in soa if f["aplica"]]
        brechas = [
            {"control": f["control"], "nombre": f["nombre"], "estado":
f["estado"],
             "solicitar_a": f["responsable"]}
            for f in aplicables if f["estado"] != "implementado"
        ]
        resumen = {
            "controles_evaluados": len(soa),
            "aplican": len(aplicables),
            "excluidos": len(soa) - len(aplicables),
            "implementados": sum(f["estado"] == "implementado" for f in
aplicables),
            "parciales": sum(f["estado"] == "parcial" for f in
aplicables),
            "no_implementados": sum(f["estado"] == "no implementado"
for f in aplicables),
        }
        org = perfil.get("organizacion", "la organización")
        texto = (
            f"SoA generada para {org} (rol: {rol}). Controles
evaluados: "
            f"{resumen['controles_evaluados']}; aplican:
{resumen['aplican']}; "
            f"implementados: {resumen['implementados']}; brechas:
{len(brechas)}. "
            "Requiere revisión y aprobación humana."
        )
        return self._resp(
            "generar_soa", "ok", texto,
            datos={"soa": soa, "brechas": brechas, "resumen": resumen},
            fuentes=["catalogo_controles.json"], confianza=1.0,
revision=True,
        )

    def _verificar_control(self, entrada):
        cid = entrada.get("control")
        control = next((c for c in self.catalogo if c["id"] == cid),
None)
        if control is None:
            raise ValueError(f"el control '{cid}' no está en el
catálogo")
        rol = (entrada.get("perfil") or {}).get("rol")
        if rol not in ROLES:
            raise ValueError("perfil.rol debe ser 'responsable',
'encargado' o 'ambos'")

        fila = self._fila(control, rol, entrada.get("riesgos", []),
```

```
entrada.get("evidencias", []))
        fila["recomendacion"] = self._recomendar(fila)
        texto = f"Control {fila['control']} ({fila['nombre']}):
{fila['estado']}. {fila['recomendacion']}"
        return self._resp(
            "verificar_control", "ok", texto, datos=fila,
            fuentes=["catalogo_controles.json"], confianza=1.0,
        )

    @staticmethod
    def _recomendar(fila):
        if not fila["aplica"]:
            return "Documentar la exclusión y su justificación en la
SoA."
        if fila["estado"] == "implementado":
            return "Mantener la evidencia vigente para la auditoría
interna (cláusula 9.2)."
        if fila["estado"] == "parcial":
            return f"Completar la implementación; coordinar con el
{fila['responsable']}."
        return f"Solicitar evidencia o implementación al
{fila['responsable']}."

    def _consultar(self, solicitud):
        if not solicitud.strip():
            return self._resp("consultar", "rechazada", "La consulta
está vacía.")
        hits = self.normativa.buscar(solicitud, k=3)
        if not hits or hits[0]["score"] < UMBRAL_CONSULTA:
            return self._resp(
                "consultar", "sin_respuesta",
                "No tengo información suficiente en mi base de
conocimiento para responder.",
                confianza=hits[0]["score"] if hits else 0.0,
            )
        return self._resp(
            "consultar", "ok", "\n\n".join(h["texto"] for h in hits),
            fuentes=sorted({h["meta"] for h in hits}),
            confianza=round(hits[0]["score"], 3),
        )

    @staticmethod
    def _resp(operacion, estado, respuesta, datos=None, fuentes=None,
              confianza=0.0, revision=False):
        return {
            "agente": "controles",
            "operacion": operacion,
            "estado": estado,
            "respuesta": respuesta,
            "datos": datos or {},
            "fuentes": fuentes or [],
            "confianza": confianza,
```

```
    "requiere_revision_humana": revision,
}
```